



FUTURE INTERNS

Cyber Security Internship 2026

Professional Application Security Assessment Report

Prepared By	Yashvardhan Patel
Program	Future Interns – Cyber Security Internship 2026
Report Type	API Security Assessment
Report Date	May 2026
Classification	CONFIDENTIAL
Version	1.0 – Final

This report is prepared solely for educational and portfolio purposes as part of the Future Interns Cyber Security Internship 2026. All testing was conducted ethically on publicly available demo APIs. No systems were exploited or compromised.

TABLE OF CONTENTS

01	Executive Summary	3
02	Project Objective	3
03	Scope & Ethical Guidelines	4
04	APIs Tested	4
05	Tools Used	5
06	Methodology	5
07	API Testing Results	6
08	Security Findings	7
09	Positive Security Controls Observed	9
10	OWASP API Security Top 10 Mapping	10
11	Risk Severity Matrix	11
12	Remediation Recommendations	11
13	Conclusion	13
14	Disclaimer	13

01 EXECUTIVE SUMMARY

This report presents the findings of a professional API Security Risk Analysis conducted as part of the Future Interns Cyber Security Internship 2026. The assessment was performed by Yashvardhan Patel using industry-standard ethical testing methodologies against two publicly available demonstration APIs: **JSONPlaceholder** (jsonplaceholder.typicode.com) and **ReqRes** (reqres.in).

The primary objective was to evaluate the security posture of these APIs from an Application Security (AppSec) perspective, identify potential risk vectors, and document observable security controls — all without conducting any form of exploitation, bypass, or unauthorized access. Testing was strictly limited to read-only, ethical, and non-destructive techniques consistent with responsible disclosure and security research best practices.

9	3	1	5
Total Findings	High Severity	Medium Severity	Informational

The assessment identified **three (3) HIGH severity findings**, one (1) MEDIUM severity finding, and five (5) informational observations reflecting positive security controls. The most significant risks relate to missing authentication, excessive data exposure, and potential IDOR patterns on the JSONPlaceholder API — all by design for a demo platform, yet serving as critical educational examples of real-world API security weaknesses.

02 PROJECT OBJECTIVE

The objectives of this API Security Risk Analysis are as follows:

- Apply ethical, read-only API security testing techniques to publicly available demonstration APIs.
- Identify and document API security risks mapped to the OWASP API Security Top 10 framework.
- Evaluate the presence of positive security controls such as authentication enforcement, rate limiting, and security headers.
- Develop professional security findings with business impact analysis and actionable remediation recommendations.
- Demonstrate applied Application Security (AppSec) skills suitable for a cybersecurity professional portfolio.
- Produce a client-grade security assessment report reflecting industry-standard consulting practices.

03

SCOPE & ETHICAL GUIDELINES

Assessment Scope

This assessment is strictly scoped to the following publicly accessible, intentionally open demonstration APIs. No internal, proprietary, or production systems were targeted, accessed, or interacted with at any point during this engagement.

Target	URL	Type	Authorization
JSONPlaceholder	https://jsonplaceholder.typicode.com	Public Demo API	Public / No Auth Required
ReqRes	https://reqres.in	Public Demo API	Public / API Key Demo

Ethical Testing Commitments

- **Read-Only Testing:** All API interactions were strictly read-only (HTTP GET requests). No POST, PUT, PATCH, DELETE, or other mutating operations were performed.
- **No Exploitation Attempted:** No attempts were made to bypass authentication controls, exploit identified vulnerabilities, exfiltrate data beyond observation, or cause service disruption.
- **Public APIs Only:** All tested endpoints are intentionally public and designed for demonstration and educational purposes. No credentials, tokens, or authorization bypasses were used.
- **Responsible Conduct:** Testing followed the spirit of responsible disclosure and ethical hacking frameworks including OWASP, EC-Council CEH guidelines, and applicable cybersecurity research ethics.
- **No Unauthorized Access:** At no point during this assessment were unauthorized systems accessed, private data harvested, or security controls actively circumvented.

04

APIS TESTED

JSONPlaceholder API

URL: <https://jsonplaceholder.typicode.com>

JSONPlaceholder is a free, open REST API designed for testing and prototyping. It provides fake data across a range of endpoints (users, posts, comments, albums, photos, todos) and intentionally lacks authentication — making it an ideal target for demonstrating API security risks in a safe, ethical, educational context.

Tested Endpoints: /users, /users/{id}, /posts, /posts/{id}, /comments

Authentication: None (Public) | **Protocol:** HTTPS

ReqRes API**URL:** <https://reqres.in>

ReqRes is a hosted REST API that simulates a real-world service including user registration, login, and resource management. It demonstrates proper API security controls such as API key enforcement, HTTP 401 Unauthorized responses, and strong security header configurations. ReqRes was assessed to evaluate positive security control implementation.

Tested Endpoints: /api/users, /api/users/{id}, /api/login, /api/register**Authentication:** API Key Required (Demo) | **Protocol:** HTTPS

05 TOOLS USED

Tool	Purpose
curl	Command-line HTTP client used to send API requests, inspect response bodies, and analyze HTTP headers.
jq	Lightweight JSON processor used to parse and format API responses for structured analysis.
Firefox DevTools	Browser-based developer tools used to inspect API calls, headers, cookies, and network traffic.
Kali Linux	Industry-standard penetration testing operating system used as the primary assessment environment.
GitHub	Used for version control, documentation, and portfolio publication of all testing artifacts and findings.

06 METHODOLOGY

The assessment followed a structured, phase-based API security testing methodology aligned with OWASP API Security Testing Guidelines and industry-standard Application Security (AppSec) practices:

01	Phase 1 — Reconnaissance & API Discovery Identified and catalogued all publicly exposed API endpoints using documentation review and direct HTTP enumeration. Baseline request/response behavior was documented for each endpoint.
02	Phase 2 — Authentication & Authorization Analysis Evaluated each endpoint for authentication requirements. Tested whether protected resources were accessible without credentials. Analyzed HTTP response codes for proper unauthorized handling.
03	Phase 3 — Data Exposure Analysis Inspected all API responses for sensitive data fields including PII (email, phone, address, geolocation). Evaluated whether field-level data minimization principles were applied.
04	Phase 4 — IDOR & Object Reference Analysis Examined URL patterns and object identifiers for sequential ID usage that could indicate Insecure Direct Object Reference (IDOR) vulnerabilities.
05	Phase 5 — Security Header Inspection Analyzed HTTP response headers for security-critical configurations including Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, and rate limiting headers.
06	Phase 6 — Rate Limiting Assessment Evaluated the presence and effectiveness of rate limiting controls by inspecting rate limit response headers and API behavior under repeated requests.

07

Phase 7 — Findings Documentation & Risk Rating

All findings were classified by severity (High / Medium / Low / Informational) using a risk-based scoring approach and mapped to the OWASP API Security Top 10 2023 framework.

07 API TESTING RESULTS

The following subsections present the raw testing evidence collected during the assessment, including curl commands executed and representative API responses observed.

7.1 — Missing Authentication Test (JSONPlaceholder)

A GET request was issued to the /users endpoint without any authentication token, API key, or authorization header. The API returned a full user dataset with no access restriction.

Command Executed:

```
$ curl -I https://jsonplaceholder.typicode.com/users
HTTP/2 200
content-type: application/json; charset=utf-8
x-content-type-options: nosniff
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
access-control-allow-credentials: true
```

Observation: Full HTTP 200 OK response returned without any authentication requirement. User data including email, phone, address, and geolocation was returned in plain JSON.

7.2 — Excessive Data Exposure Test (JSONPlaceholder)

Inspection of the /users endpoint response body revealed multiple sensitive PII fields returned by default without any field-level filtering or data minimization.

Representative Response Fragment:

```
$ curl https://jsonplaceholder.typicode.com/users/1 | jq .
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
  "phone": "1-770-736-0988 x56442",
  "company": { "name": "Romaguera-Crona" }
}
```

7.3 — IDOR Pattern Analysis (Sequential IDs)

API endpoints were observed to use sequential integer-based resource identifiers, a pattern that may facilitate Insecure Direct Object Reference (IDOR) attacks in authenticated production systems.

```
$ curl https://jsonplaceholder.typicode.com/users/1
$ curl https://jsonplaceholder.typicode.com/users/2
$ curl https://jsonplaceholder.typicode.com/users/3
# All return distinct user records – sequential enumeration confirmed
```

7.4 — ReqRes Authentication Enforcement Test

ReqRes properly rejected unauthenticated API requests, returning a structured error response with HTTP 401 Unauthorized status.

```
$ curl -X POST https://reqres.in/api/login \
-H 'Content-Type: application/json' \
-d '{"email": "eve.holt@reqres.in"}'
HTTP/2 400
{
  "error": "Missing password"
}
$ curl https://reqres.in/api/users -H 'Authorization: Bearer invalid'
HTTP/2 401
{
  "error": "missing_api_key"
}
```

7.5 — Security Header Analysis (ReqRes)

```
$ curl -I https://reqres.in/api/users
HTTP/2 200
strict-transport-security: max-age=31536000; includeSubDomains; preload
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
cross-origin-resource-policy: same-origin
content-type: application/json; charset=utf-8
```

08

SECURITY FINDINGS

The following table summarizes all security findings identified during the assessment. Each finding is assigned a unique identifier, severity rating, business impact assessment, and targeted remediation recommendation.

ID	Finding	Severity	Affected Target	Business Impact
F-001	Missing Authentication on Public Endpoints	HIGH	JSONPlaceholder	Unauthorized access to user data; data breach risk in production-equivalent systems.
F-002	Excessive Data Exposure (PII Leakage)	HIGH	JSONPlaceholder	Email, phone, address, and geolocation exposed unnecessarily; GDPR / privacy compliance risk.
F-003	Sequential ID Enumeration — Potential IDOR Pattern	MEDIUM	JSONPlaceholder	Enables systematic user enumeration; IDOR exploitation risk in authenticated systems.
F-004	Public Exposure of Personally Identifiable Information	HIGH	JSONPlaceholder	PII returned without consent controls; regulatory, reputational, and legal exposure.
F-005	Rate Limiting Controls Present	INFO	JSONPlaceholder	Positive control — rate limiting mitigates brute force and scraping risks.
F-006	Security Headers Present (X-Content-Type-Options)	INFO	JSONPlaceholder	Positive control — MIME-type sniffing attack surface reduced.
F-007	API Key Authentication Enforcement	INFO	ReqRes	Positive control — unauthenticated requests properly rejected with 401.
F-008	Proper HTTP 401 Unauthorized Handling	INFO	ReqRes	Positive control — consistent and correct error messaging for unauthorized access.
F-009	Strong Security Header Configuration	INFO	ReqRes	Positive control — HSTS, X-Frame-Options, CORP headers reduce attack surface.

High Severity Finding Detail: F-001 — Missing Authentication

F-001 | Missing Authentication on Public Endpoints | SEVERITY: HIGH

Description: The JSONPlaceholder /users endpoint returns complete user records without requiring any form of authentication. Any anonymous client can retrieve the full user dataset by issuing a standard HTTP GET request.

OWASP Mapping: API2:2023 — Broken Authentication

CVSS Indicator: High — Confidentiality impact to exposed user data.

Recommendation: Implement Bearer token or OAuth 2.0 authentication on all endpoints that return user-identifiable data. Apply the principle of least privilege to API endpoint access control.

High Severity Finding Detail: F-002 — Excessive Data Exposure

F-002 | Excessive Data Exposure (PII Leakage) | SEVERITY: HIGH

Description: API responses return fields including email address, phone number, home address, geographic coordinates, and company information without any field-level filtering or data minimization controls.

Exposed PII Fields: email, phone, address.street, address.city, address.geo.lat, address.geo.lng, company.name, company.catchPhrase

OWASP Mapping: API3:2023 — Broken Object Property Level Authorization

Recommendation: Implement response field filtering. Return only fields required for the requesting client's legitimate use case. Apply field-level authorization to restrict sensitive attributes.

09

POSITIVE SECURITY CONTROLS OBSERVED

The following positive security controls were identified during the assessment. These represent correctly implemented security measures that reduce risk and should be maintained and extended as the API evolves.

✓ Rate Limiting Enforcement (JSONPlaceholder)

Rate limiting headers (x-ratelimit-limit, x-ratelimit-remaining) were observed in API responses, confirming active enforcement of request throttling. This mitigates brute-force enumeration, denial-of-service via API flood, and automated scraping attacks.

Evidence:

```
x-ratelimit-limit: 1000
```

```
x-ratelimit-remaining: 999
```

✓ API Key Authentication Enforcement (ReqRes)

ReqRes properly validates API key credentials and returns structured HTTP 401 Unauthorized responses for unauthenticated or incorrectly authenticated requests. The error response includes a meaningful error field (missing_api_key) without leaking sensitive stack traces.

Evidence:

```
{"error": "missing_api_key"} – HTTP 401 Unauthorized
```

✓ Strong Security Header Configuration (ReqRes)

ReqRes implements a comprehensive set of HTTP security headers that significantly reduce the attack surface for common web and API attacks:

Evidence:

```
strict-transport-security: max-age=31536000; includeSubDomains; preload
```

```
x-frame-options: SAMEORIGIN
```

```
x-content-type-options: nosniff
```

```
cross-origin-resource-policy: same-origin
```

10 OWASP API SECURITY TOP 10 MAPPING

All findings have been mapped to the OWASP API Security Top 10 2023 framework, providing standardized risk categorization aligned with industry-recognized security guidance.

OWASP ID	Category	Finding(s)	Status
API1:2023	Broken Object Level Authorization	F-003: Sequential ID enumeration enabling potential IDOR	IDENTIFIED
API2:2023	Broken Authentication	F-001: No auth required on sensitive endpoints; F-007: Auth correctly enforced on ReqRes	IDENTIFIED / POSITIVE
API3:2023	Broken Object Property Level Auth	F-002: Excessive PII fields returned without field-level filtering	IDENTIFIED
API4:2023	Unrestricted Resource Consumption	F-005: Rate limiting present and enforced on JSONPlaceholder	POSITIVE CONTROL
API8:2023	Security Misconfiguration	F-006, F-008, F-009: Security headers verified; 401 handling correct on ReqRes	POSITIVE CONTROL

11 RISK SEVERITY MATRIX

The risk severity matrix below provides a visual overview of all findings, classified by likelihood and business impact. Severity ratings are assigned based on observed evidence, potential for real-world exploitation, and regulatory exposure.

Severity	Findings	Count	Priority
HIGH	F-001 Missing Authentication, F-002 Excessive Data Exposure, F-004 PII Exposure	3	Immediate Remediation Required
MEDIUM	F-003 Sequential ID Enumeration (IDOR Pattern)	1	Remediate Within 30 Days
INFO	F-005 Rate Limiting, F-006 Security Header (JSONPlaceholder), F-007 API Key Auth, F-008 HTTP 401 Handling, F-009 Strong Headers (ReqRes)	5	Maintain & Monitor

12 REMEDIATION RECOMMENDATIONS

The following remediation recommendations are provided to address identified security findings. Recommendations are prioritized by severity and aligned with OWASP API Security best practices. All recommendations are provided for educational and portfolio demonstration purposes.

REC-001 | Implement Authentication on All Sensitive Endpoints | HIGH

Deploy Bearer token authentication (JWT) or OAuth 2.0 on all endpoints that return user-identifiable or sensitive data. Unauthenticated requests should receive HTTP 401 Unauthorized responses with a minimal, non-leaking error body. Implement role-based access control (RBAC) to ensure clients can only access resources within their authorization scope.

Implementation Reference:

```
Authorization: Bearer <JWT_TOKEN>
Response on failure: HTTP 401 {'error': 'unauthorized'}
```

REC-002 | Apply Data Minimization — Reduce Exposed PII Fields | HIGH

Implement field-level response filtering to ensure only fields necessary for the requesting client's use case are returned. Sensitive fields such as geolocation, home address, and phone number should require elevated authorization. Consider adopting a 'deny by default' approach to field exposure, following GDPR data minimization principles and OWASP API3 guidance.

Implementation Reference:

```
# Return only: id, name, username — not email/phone/address
# Use field projection or response DTO serialization
GET /users/1 -> {'id': 1, 'name': 'John Doe', 'username': 'jdoe'}
```

REC-003 | Replace Sequential IDs with UUID-Based Identifiers | MEDIUM

Replace integer-based sequential resource identifiers (/users/1, /users/2) with randomly generated UUIDs (Universally Unique Identifiers). This eliminates the predictability that enables IDOR enumeration attacks, significantly raising the effort required for object reference guessing even when authentication is present.

Implementation Reference:

```
# Before: GET /users/1
# After: GET /users/a1b2c3d4-e5f6-7890-abcd-ef1234567890
import uuid; user_id = str(uuid.uuid4())
```

REC-004 | Implement Field-Level Authorization Controls | MEDIUM

Beyond authentication, apply attribute-level access controls so that even authenticated users can only access fields appropriate to their role. An 'admin' role might see email and phone, while a standard 'user' role sees only name and username. This prevents privilege escalation via API response inspection.

Implementation Reference:

```
# Role-based field filtering pseudocode
if user.role == 'admin': return full_user_object
else: return {'id': user.id, 'name': user.name}
```

REC-005 | Continue Enforcing Rate Limiting | INFO

Rate limiting is correctly implemented and should be maintained and extended across all API endpoints. Consider implementing per-user rate limiting in addition to global limits, and return Retry-After headers to improve API client usability. Monitor rate limit logs for anomalous activity patterns.

Implementation Reference:

```
x-ratelimit-limit: 1000
x-ratelimit-remaining: <current>
x-ratelimit-reset: <epoch_timestamp>
Retry-After: 60 # On 429 Too Many Requests
```

REC-006 | Maintain and Expand Strong Security Header Configuration | INFO

The current security header implementation on ReqRes is exemplary and should serve as the baseline for all API deployments. Additionally, consider adding the following headers: Content-Security-Policy, Permissions-Policy, and Referrer-Policy. Conduct quarterly security header audits using tools such as securityheaders.com.

Implementation Reference:

```
strict-transport-security: max-age=31536000; includeSubDomains; preload
x-frame-options: DENY
x-content-type-options: nosniff
content-security-policy: default-src 'none'
referrer-policy: no-referrer
```

REC-007 | Continue API Key Validation Enforcement | INFO

The API key authentication enforcement on ReqRes correctly rejects invalid or missing credentials with structured, informative-yet-safe error responses. This pattern should be documented as an internal security standard and applied consistently across all protected API resources. Implement API key rotation policies and key expiry mechanisms.

Implementation Reference:

```
# Correct pattern – do not leak implementation details
HTTP 401 {'error': 'missing_api_key'}
HTTP 401 {'error': 'invalid_api_key'}
# Avoid: HTTP 500 with stack traces
```

13 CONCLUSION

This API Security Risk Analysis successfully demonstrates the application of professional Application Security methodologies to publicly available demonstration APIs. The assessment identified three HIGH severity findings, one MEDIUM severity finding, and five informational positive security control observations — all consistent with realistic, real-world API security posture evaluation.

The JSONPlaceholder API, while intentionally open by design, clearly illustrates the security risks that arise in production systems when authentication is absent and data minimization principles are not applied. These findings serve as concrete, evidence-based examples of OWASP API2 (Broken Authentication) and API3 (Broken Object Property Level Authorization) vulnerabilities.

Conversely, the ReqRes API demonstrates commendable implementation of API security best practices — proper authentication enforcement, structured 401 responses, and a strong security header configuration that aligns with HSTS preloading standards. ReqRes serves as a reference implementation of production-grade API security controls.

The remediation recommendations provided in this report are actionable, prioritized, and aligned with industry frameworks including OWASP API Security Top 10 2023, NIST SP 800-95, and general API security engineering best practices. Implementation of the HIGH and MEDIUM priority recommendations would substantially improve the security posture of equivalent production systems.

Assessment Summary

- All testing was conducted ethically using read-only techniques on public demo APIs.
- No systems were exploited, compromised, or accessed without authorization.
- Findings are documented for educational, portfolio, and internship assessment purposes.
- This report follows industry-standard security consulting report structure and language.

14 DISCLAIMER

Legal & Ethical Disclaimer

This report has been prepared exclusively for educational purposes as part of the Future Interns Cyber Security Internship 2026 program. All API security testing documented herein was conducted against publicly available, free, demonstration APIs (JSONPlaceholder and ReqRes) that are explicitly designed for testing purposes.

No unauthorized access was performed. No production systems, private APIs, or third-party infrastructure were targeted, accessed, or affected. All testing was strictly read-only, non-exploitative, and consistent with ethical security research principles and responsible disclosure standards.

The findings, severity ratings, and risk assessments presented in this report are based on observable behavior of demonstration APIs and are intended solely to illustrate API security concepts. They do not constitute formal security audit findings, legal advice, or security certifications of any kind.

The author, Yashvardhan Patel, bears no liability for the use or misuse of information contained in this report. This document is intended for GitHub portfolio showcase, LinkedIn publication, and internship submission only.